



UNIVERSIDADE
ESTADUAL de LONDRINA

ANA LAURA MACHADO ARAUJO CAETANO
LAURA TAMAROZZI

APACHE SPARK

*Trabalho de Sistemas Distribuídos apresentado ao
Departamento de Computação da Universidade
Estadual de Londrina como requisito parcial para
aprovação na disciplina.*

Professor(a): Fabio Sakuray

RESUMO

Este trabalho apresenta a análise de dados climáticos globais utilizando Apache Spark em um cluster distribuído provisionado com Docker. Foram processados dois conjuntos de dados: o GlobalLandTemperaturesByCity.csv (Berkeley Earth / Kaggle), com 8.599.212 registros de temperaturas mensais por cidade desde 1750, e o owid-co2-data.csv (Our World in Data), com emissões de CO2 por país e ano. O pipeline ETL reduziu os dados para 4.746.210 registros após limpeza, com ganho de velocidade de 34x via cache. Foram respondidas 8 perguntas analíticas cobrindo tendências de aquecimento, anomalias climáticas, instabilidade térmica, correlações estatísticas e previsão com Spark MLlib.

Palavras-chave: Apache Spark. Big Data. Análise Climática. Processamento Distribuído. ETL.

1. INTRODUÇÃO

As mudanças climáticas representam um dos maiores desafios científicos e sociais do século XXI. O volume de dados históricos disponíveis sobre temperatura global, precipitação e emissões de gases de efeito estufa cresce continuamente, demandando ferramentas de processamento distribuído capazes de lidar com dezenas de milhões de registros de forma eficiente.

Este projeto tem como objetivo demonstrar a superioridade do Apache Spark — framework de processamento em memória para Big Data — no tratamento de dados climáticos em larga escala. Utilizando o dataset Berkeley Earth com registros de temperatura de 1750 até o presente, o trabalho responde a oito questões analíticas que cobrem desde tendências de aquecimento por década até previsão de temperatura com regressão linear via Spark MLlib.

2. ARQUITETURA DO CLUSTER E AMBIENTE DISTRIBUÍDO

1.1 2.1 VISÃO GERAL DA ARQUITETURA

A arquitetura do projeto segue o modelo mestre-trabalhador (Master/Worker), padrão do Apache Spark Standalone, em que um nó central (Master) coordena a distribuição de tarefas e os nós trabalhadores (Workers) executam o processamento paralelo. Todo o cluster foi containerizado com Docker e orquestrado via Docker Compose.

O fluxo de dados segue o seguinte caminho: o Jupyter Lab (interface de desenvolvimento) conecta-se ao Spark Master via porta 7077. O Master distribui as tarefas entre os dois Workers, que processam os dados em paralelo e retornam os resultados ao Driver. Os volumes Docker mapeiam as pastas locais data/, notebooks/ e output/ para dentro dos containers, tornando os arquivos acessíveis a todos os serviços simultaneamente.

2.2 COMPONENTES DO CLUSTER

O cluster é composto por quatro containers Docker com responsabilidades distintas:

Tabela 1 — Configuração detalhada dos containers do cluster Spark

Container	Imagem Docker	Função	Memória	Porta(s)
spark-master	apache/spark-py:v3.5.1	Coordena workers e distribui tarefas	—	8080 (UI), 7077 (cluster)
spark-worker-1	apache/spark-py:v3.5.1	Executa partições do RDD/DataFrame	2 GB	8081 (UI)
spark-worker-2	apache/spark-py:v3.5.1	Executa partições do RDD/DataFrame	2 GB	8082 (UI)
jupyter-spark	jupyter/pyspark-notebook:3.5.0	Interface de desenvolvimento (Driver)	4 GB	8888 (Lab), 4040 (Spark)

O container jupyter-spark atua como Driver da aplicação Spark: é ele que envia o plano de execução ao Master e recebe os resultados agregados. Os Workers recebem as Tasks do Master e operam sobre as partições do DataFrame de forma paralela e independente. Com 2 Workers de 2 cores cada, o cluster dispõe de 4 Executor Cores simultâneos para processamento paralelo.

2.3 CONFIGURAÇÃO DO DOCKER COMPOSE

O arquivo docker-compose.yml define todos os serviços, variáveis de ambiente, portas expostas e volumes compartilhados. Os volumes garantem que os datasets em data/ e os notebooks em notebooks/ sejam acessíveis tanto pelo Jupyter quanto pelos Workers, sem necessidade de cópia entre containers:

```
services:
  spark-master:
    image: apache/spark-py:v3.5.1
    ports: ["8080:8080", "7077:7077"]
    volumes:
      - ./data:/data
      - ./notebooks:/notebooks
      - ./output:/output

  spark-worker-1:
    image: apache/spark-py:v3.5.1
    environment:
      - SPARK_WORKER_MEMORY=2g
      - SPARK_WORKER_CORES=2
    depends_on: [spark-master]

  jupyter:
    image: jupyter/pyspark-notebook:spark-3.5.0
    environment:
      - SPARK_MASTER=spark://spark-master:7077
    ports: ["8888:8888", "4040:4040"]
```

2.4 CONFIGURAÇÃO DA SPARKSESSION E PARÂMETROS DE EXECUÇÃO

A SparkSession é o ponto de entrada da aplicação Spark. No projeto, foi configurada no modo local[*], que utiliza todos os núcleos disponíveis da máquina como Executors dentro de um único processo JVM. Essa escolha é adequada para desenvolvimento e prototipagem, eliminando a sobrecarga de comunicação em rede

entre processos distintos:

```
spark = SparkSession.builder \
    .master("local[*]") \
    .appName("ClimateAnalysis") \
    .config("spark.driver.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "50") \
    .getOrCreate()
```

Tabela 2 — Parâmetros de configuração da SparkSession e seu impacto

Parâmetro	Valor	Justificativa
master	local[*]	Usa todos os núcleos da CPU como Executors no mesmo processo
spark.driver.memory	4g	Memória do Driver para coletar resultados agregados e rodar toPandas()
spark.sql.shuffle.partitions	50	Reduz as 200 partições padrão para o volume do dataset (~4,7M linhas), diminuindo overhead de shuffle
appName	ClimateAnalysis	Identifica a aplicação no Spark UI (http://localhost:4040)

2.5 INTERFACES DE MONITORAMENTO

O Apache Spark disponibiliza interfaces web para monitoramento em tempo real do cluster e das aplicações. Essas interfaces foram utilizadas para capturar as evidências do Spark UI exigidas pelo relatório:

Interface	URL	O que mostra
Spark Master UI	http://localhost:8080	Estado do cluster: workers conectados, recursos disponíveis e jobs ativos
Spark Application UI	http://localhost:4040	DAG de execução, stages, tasks, tempo por operação e uso de memória
Spark UI — Storage	http://localhost:4040/storage	DataFrames em cache: tamanho em memória e número de partições
Spark UI — Stages	http://localhost:4040/stages	Detalhe de cada estágio: tempo, shuffle read/write e tasks concluídas
Worker 1 UI	http://localhost:8081	Recursos e tasks em execução no worker 1
Worker 2 UI	http://localhost:8082	Recursos e tasks em execução no worker 2
Jupyter Lab	http://localhost:8888	Interface de desenvolvimento e execução dos notebooks

2.6 FLUXO DE EXECUÇÃO DISTRIBUÍDA

Quando uma célula do notebook é executada, o Spark traduz o código Python em um plano de execução lógico e depois físico, seguindo estas etapas:

Etapa	Fase	Descrição
1	Parsing	O Driver converte as transformações PySpark em um DAG (Grafo Acíclico Dirigido) de operações.
2	Otimização (Catalyst)	O Catalyst Optimizer reordena e simplifica as operações para minimizar leitura de dados e shuffle.
3	Divisão em Stages	O DAG é dividido em Stages separados por operações de shuffle (ex: groupBy, join).
4	Divisão em Tasks	Cada Stage é dividido em Tasks — uma por partição do DataFrame — distribuídas aos Executors.
5	Execução paralela	Os Executors (Workers) processam suas partições simultaneamente, em memória.
6	Coleta do resultado	O Driver coleta os resultados parciais, agrega e retorna ao notebook.

O modelo de execução lazy evaluation (avaliação preguiçosa) do Spark significa que as transformações como `filter()`, `withColumn()` e `groupBy()` não são executadas imediatamente — apenas quando uma ação como `count()`, `show()` ou `toPandas()` é chamada. Isso permite ao Catalyst Optimizer otimizar o plano completo antes de qualquer processamento, reduzindo operações desnecessárias.

2.7 GERENCIAMENTO DE VOLUMES E DADOS

Os dados transitam entre os containers via volumes Docker mapeados. Essa arquitetura garante que qualquer alteração nos arquivos locais seja imediatamente refletida dentro dos containers, sem necessidade de rebuild da imagem:

Pasta local	Mapeamento no container	Conteúdo
<code>./data/</code>	<code>/home/jovyan/data/</code> (Jupyter), <code>/data/</code> (Workers)	Datasets brutos: GlobalLandTemperaturesByCity.csv e owid-co2-data.csv

./notebooks/	/home/jovyan/work/ (Jupyter)	Notebooks .ipynb com o pipeline completo
./output/	/home/jovyan/output/ (Jupyter)	Gráficos .png gerados e dataset final .parquet

3. METODOLOGIA ETL

3.1 DATASETS UTILIZADOS

O projeto utiliza dois conjuntos de dados complementares:

Tabela 2 — Datasets utilizados no projeto

Dataset	Fonte	Registros Brutos	Granularidade
GlobalLandTemperaturesByCity.csv	Berkeley Earth / Kaggle	8.599.212	Cidade / Mês
owid-co2-data.csv	Our World in Data	~50.000	País / Ano

3.2 PIPELINE DE LIMPEZA

O processo de ETL seguiu o fluxo: Leitura → Limpeza → Enriquecimento → Processamento → Saída. Cada etapa foi implementada como uma transformação imutável no DataFrame Spark, garantindo rastreabilidade e não alterando os arquivos originais.

Tabela 3 — Etapas do pipeline de limpeza

Etapa	Operação	Código PySpark	Justificativa
A	Remoção de nulos	df.dropna(subset=["AverageTemperature"])	Elimina meses sem medição
B	Conversão de datas	to_date() + year() + month()	Habilita groupBy por ano/mês/década
C	Filtro de incerteza	uncertainty < 1.5°C	Remove medições com margem de erro alta
D	Filtro de período	Year >= 1900	Foca no período com dados mais densos
E	Limpeza de coordenadas	regexp_replace() N/S/E/W → numérico	Necessário para filtro tropical (P4)
F	Normalização de strings	trim() nos campos Country e City	Evita duplicatas por espaço extra

G	Cache	df_clean.cache()	Persiste em memória para as 8 consultas
---	-------	------------------	---

3.3 RESULTADO DA LIMPEZA

Após a aplicação de todas as transformações, o dataset foi reduzido de 8.599.212 para 4.746.210 registros, representando uma retenção de 55,2% dos dados originais. Os registros descartados correspondiam principalmente a medições anteriores a 1900 e leituras com incerteza superior a 1,5°C.

4. ANÁLISE DE OTIMIZAÇÃO COM CACHE

Uma das principais vantagens do Apache Spark é a capacidade de persistir DataFrames em memória, evitando reprocessamento do pipeline ETL a cada consulta. O método `.cache()` foi aplicado ao DataFrame limpo após todas as transformações.

Tabela 4 — Comparativo de tempo com e sem cache

Execução	Tempo	Observação
1ª varredura (SEM cache)	16,30s	Lê CSV + aplica todas as transformações
2ª varredura (COM cache)	0,48s	Recupera direto da memória RAM
Ganho de velocidade	34x mais rápido	Benefício crescente com múltiplas consultas

O ganho de 34x demonstra a eficiência do modelo de execução em memória do Spark. Em um cenário com 8 consultas analíticas sobre o mesmo DataFrame, o cache evitou 7 releituras completas do CSV, reduzindo drasticamente o tempo total de execução.

5. RESULTADOS

5.1 PERGUNTA 1 — TEMPERATURA MÉDIA GLOBAL POR DÉCADA

Resposta direta: A temperatura média global aumentou de 17,49°C na década de 1900 para 18,63°C na década de 2010, um aumento total de 1,14°C em 110 anos. O crescimento foi gradual até 1970, acelerando significativamente a partir dos anos 1980.

Tabela 5 — Temperatura média global por década (1900–2010)

Década	Temp. Média (°C)	Variação vs. anterior
1900	17,49	—
1910	17,46	-0,03°C
1920	17,63	+0,17°C
1930	17,75	+0,12°C
1940	17,78	+0,03°C
1950	17,80	+0,02°C
1960	17,78	-0,02°C
1970	17,80	+0,02°C
1980	17,96	+0,16°C
1990	18,25	+0,29°C
2000	18,52	+0,27°C
2010	18,63	+0,11°C

Interpretação: O padrão de aquecimento é consistente com o fenômeno do aquecimento global. A aceleração observada a partir da década de 1980 coincide com o período de maior industrialização global e aumento das emissões de CO₂.

```
# Código utilizado — Pergunta 1
df_decade = df_clean \
    .groupBy("Decade") \
    .agg(F.round(F.avg("AverageTemperature"), 2).alias("AvgTemp")) \
    .orderBy("Decade")
```

5.2 PERGUNTA 2 — 10 ANOS MAIS QUENTES POR CONTINENTE (ÚLTIMOS 50 ANOS)

Resposta direta: Para a América do Norte, o ano mais quente foi 2013

(17,18°C). Para a América do Sul, os anos mais quentes concentram-se na década de 2000 e 2010. A análise utilizou Window Functions com rank() particionado por continente.

Tabela 6 — Top 5 anos mais quentes — América do Norte e América do Sul

Continente	Rank	Ano	Temp. Média (°C)
América do Norte	1	2013	17,18
América do Norte	2	2012	16,66
América do Norte	3	1998	16,32
América do Norte	4	2006	16,30
América do Norte	5	2005	16,05

```
# Código utilizado — Pergunta 2
window_cont = Window.partitionBy("Continent").orderBy(F.desc("AvgTemp"))
df_top10 = df_cont_year \
    .withColumn("Rank", F.rank().over(window_cont)) \
    .filter(F.col("Rank") <= 10)
```

5.3 PERGUNTA 3 — CIDADES COM MAIOR INSTABILIDADE TÉRMICA

Resposta direta: As cidades com maior desvio padrão de temperatura estão concentradas na Rússia e norte da China, regiões de clima continental extremo. Blagoveshchensk (Rússia) e Heihe (China) lideram com desvio padrão de 16,34°C, indicando amplitudes térmicas anuais muito elevadas.

Tabela 7 — Top 10 cidades com maior desvio padrão de temperatura

Cidade	País	Desvio Padrão (°C)	Registros
Blagoveshchensk	Rússia	16,34	1.077
Heihe	China	16,34	1.077
Hailar	China	16,18	1.072
Yakeshi	China	16,18	1.072
Kyzyl	Rússia	16,01	1.026
Nehe	China	15,96	1.069
Beian	China	15,93	1.069
Qiqihar	China	15,90	1.071
Qitaihe	China	15,88	1.072
Harbin	China	15,85	1.071

```
# Código utilizado – Pergunta 3
df_stddev = df_clean \
    .groupBy("City", "Country") \
    .agg(F.round(F.stddev("AverageTemperature"), 4).alias("TempStdDev")) \
    .orderBy(F.desc("TempStdDev")).limit(20)
```

5.4 PERGUNTA 4 — CORRELAÇÃO TEMPERATURA MÍNIMA VS. MÁXIMA EM ZONAS TROPICAIS

Resposta direta: O coeficiente de correlação de Pearson encontrado foi 0,4534, indicando correlação moderada a fraca entre o aumento da temperatura mínima e da temperatura máxima em zonas tropicais (latitudes entre -23,5° e +23,5°).

Interpretação: A correlação moderada sugere que as temperaturas mínimas e máximas em regiões tropicais não se movem de forma tão interdependente quanto em regiões temperadas. Isso pode refletir a maior variabilidade sazonal em alguns países tropicais com estações secas e chuvosas bem definidas.

```
# Código utilizado – Pergunta 4
df_tropical = df_clean.filter(
    (F.col("Lat") >= -23.5) & (F.col("Lat") <= 23.5))
corr = df_minmax.stat.corr("MinTemp", "MaxTemp")
# Resultado: r = 0.4534
```

5.5 PERGUNTA 5 — QUALIDADE DOS DADOS

Resposta direta: A média histórica global de temperatura é 16,73°C, portanto o limiar de 10% corresponde a 1,67°C de incerteza. Um total de 1.510.031 registros (17,6% do dataset bruto) apresentaram incerteza superior a esse limiar, sendo a maioria concentrada entre 1750 e 1850.

Tabela 8 — Registros com incerteza > 10% da média histórica por período

Período	Registros problemáticos	Observação
1700–1749	5.936	Poucos registros, alta incerteza
1750–1799	449.951	Início da medição sistemática
1800–1849	702.635	Pico de registros problemáticos
1850–1899	325.427	Melhoria gradual dos instrumentos
1900–1949	22.487	Padronização dos sensores
1950–1999	2.374	Tecnologia moderna

2000+	1.221	Raros outliers
-------	-------	----------------

```
# Código utilizado – Pergunta 5
global_mean = df_raw.agg(F.avg("AverageTemperature")).collect()[0][0]
# global_mean = 16.73°C | limiar = 1.67°C
df_issues = df_raw.filter(col("AverageTemperatureUncertainty") > threshold)
# 1.510.031 registros (17.6%)
```


5.6 PERGUNTA 6 — CORRELAÇÃO CO2 VS. TEMPERATURA (JOIN ENTRE DATASETS)

Resposta direta: O coeficiente de correlação de Pearson entre emissões de CO2 e temperatura média por país/ano foi de -0,1650, indicando correlação fraca e negativa.

Interpretação: O resultado aparentemente contra-intuitivo se explica pela natureza dos dados. Países com maiores emissões de CO2 (Rússia, Canadá, países do norte da Europa) possuem temperaturas médias anuais mais baixas por estarem em latitudes elevadas. A correlação negativa reflete essa confusão entre emissor industrial e temperatura regional, não invalidando o efeito do CO2 no aquecimento global, que é melhor observado em análise de séries temporais por país do que em um join estático.

Tabela 9 — Amostra do dataset após join Temperatura + CO2

País	Ano	Temp. Média (°C)	CO2 (Mt/ano)
China	2006	14,02	6.486,19
Russia	1990	4,79	2.536,28
Canada	1996	4,75	507,78
Turkey	2010	15,51	318,71
Spain	2006	15,98	358,87

```
# Código utilizado - Pergunta 6
df_joined = df_temp_country_year.join(
    df_co2,
    on=[temp["Country"] == co2["CO2_Country"],
        temp["Year"] == co2["Year"]], how="inner")
corr = df_joined.stat.corr("CO2_MtCO2", "AvgTemp")
# Resultado: r = -0.1650
```

5.7 PERGUNTA 7 — RANKING DE ACELERAÇÃO TÉRMICA (WINDOW FUNCTIONS)

Resposta direta: O Canadá liderou o ranking com aceleração de +0,94°C entre a década de 2000 e 2010, seguido por Iraque (+0,73°C) e Geórgia (+0,70°C). Regiões do Oriente Médio e Ásia Central dominam o top 10.

Tabela 10 — Top 10 países com maior aceleração térmica (2000s vs 2010s)

Rank	País	Temp. 2000s (°C)	Temp. 2010s (°C)	Aceleração (°C)
1°	Canadá	5,97	6,91	+0,94
2°	Iraque	21,15	21,88	+0,73
3°	Geórgia	9,61	10,31	+0,70
4°	Armênia	9,61	10,31	+0,70
5°	Irã	16,37	17,00	+0,63
6°	Turquia	13,96	14,56	+0,60
7°	Síria	19,06	19,65	+0,59
8°	Bahrein	27,13	27,69	+0,56
9°	Arábia Saudita	25,98	26,53	+0,56
10°	Laos	24,66	25,22	+0,55

```
# Código utilizado – Pergunta 7
df_pivot = df_decades.groupBy("Country").pivot("Decade").avg("AvgTemp")
df_accel = df_pivot.withColumn("TempAcceleration",
    F.col("Temp_2010s") - F.col("Temp_2000s")) \
    .orderBy(F.desc("TempAcceleration")).limit(10)
```

5.8 PERGUNTA 8 — PREVISÃO DE TEMPERATURA COM SPARK MLlib

Resposta direta: Utilizando Regressão Linear simples treinada sobre o histórico do Brasil (1993–2013), o modelo prevê temperatura média estável entre 22,93°C e 22,94°C para o período 2024–2028. O coeficiente de determinação $R^2 = 0,0086$ indica baixo poder explicativo do modelo linear simples para o Brasil.

Tabela 11 — Previsão de temperatura para o Brasil (2024–2028)

Ano	Temperatura Prevista (°C)	Modelo
2024	22,93	Regressão Linear (MLlib)
2025	22,93	Regressão Linear (MLlib)
2026	22,94	Regressão Linear (MLlib)
2027	22,94	Regressão Linear (MLlib)
2028	22,94	Regressão Linear (MLlib)

Interpretação: O baixo R^2 (0,0086) sugere que o ano isolado não é um preditor forte da temperatura média anual do Brasil, que apresenta alta variabilidade. Para resultados mais precisos, seria necessário incorporar variáveis como ENSO (El Niño/La Niña), índices de precipitação e múltiplas features no modelo MLlib.

```
# Código utilizado - Pergunta 8
assembler = VectorAssembler(inputCols=["Year"], outputCol="features")
lr = LinearRegression(featuresCol="features", labelCol="AvgTemp")
model = lr.fit(df_ml_vec)
# slope: 0.003815 °C/ano | intercept: 15.2072 | R²: 0.0086
```

6. ARTEFATOS DE SAÍDA

6.1 ARQUIVO PARQUET

O dataset final foi exportado no formato colunar Parquet, particionado pela coluna Country, resultando em 5.761 registros distribuídos em diretórios por país. O formato Parquet foi escolhido por sua eficiência de leitura em consultas analíticas e compressão superior ao CSV.

6.2 DICIONÁRIO DE DADOS

Tabela 12 — Dicionário de dados do arquivo *climate_co2_final.parquet*

Coluna	Tipo	Descrição
Country	String	Nome do país — chave de partição Parquet
Year	Integer	Ano da medição (1975–2023)
AvgTemperature_C	Double	Temperatura média anual do país em Celsius, calculada como média de todas as cidades após filtragem de incerteza < 1,5°C e remoção de nulos
CO2_Emissions_MtCO2	Double	Emissões anuais de CO2 do país em megatoneladas (fonte: Our World in Data)

7. CONCLUSÃO

Este trabalho demonstrou o poder do Apache Spark para análise de dados climáticos em larga escala. O processamento de 8,5 milhões de registros com limpeza, joins e agregações complexas foi realizado de forma eficiente em um cluster Docker local, evidenciando a viabilidade do ambiente containerizado para fins acadêmicos e de prototipagem.

Os resultados confirmam tendências documentadas pela literatura: aquecimento acumulado de 1,14°C desde 1900, aceleração a partir da década de 1980, e países do Hemisfério Norte liderando o ranking de aceleração térmica recente. O ganho de 34x com cache comprova a eficiência do modelo de execução em memória do Spark, especialmente em workflows com múltiplas consultas sobre o mesmo dataset.

Como trabalhos futuros, sugere-se a incorporação de variáveis climáticas adicionais (precipitação, ENSO), a aplicação de modelos de séries temporais mais sofisticados (ARIMA, Prophet) via Spark MLlib, e o deploy em cluster real na nuvem (AWS EMR ou Databricks) para validar a escalabilidade horizontal.

REFERÊNCIAS

BERKELEY EARTH. Climate Change: Earth Surface Temperature Data. Disponível em: <https://www.kaggle.com/datasets/berkeleyearth/climate-change-earth-surface-temperature-data>. Acesso em: mai. 2026.

OUR WORLD IN DATA. CO2 and Greenhouse Gas Emissions. Disponível em: <https://github.com/owid/co2-data>. Acesso em: mai. 2026.

APACHE SOFTWARE FOUNDATION. Apache Spark Documentation. Disponível em: <https://spark.apache.org/docs/latest/>. Acesso em: mai. 2026.

ZAHARIA, M. et al. Apache Spark: A Unified Engine for Big Data Processing. Communications of the ACM, v. 59, n. 11, p. 56–65, nov. 2016.

DOCKER INC. Docker Documentation. Disponível em: <https://docs.docker.com>. Acesso em: mai. 2026.